

MatrixLang

Checking Matrix Shapes Before They Are Computed

A Dimension-Aware Optimizing Compiler for a Matrix Language

Project Review 2: Core Implementation

BCSE307P Compiler Design Lab

School of Computer Science and Engineering, VIT Vellore

Name	Registration Number	Programme
A Aswanth Raj	24BAI0044	B.Tech CSE (AI & ML)

Supervisor	Dr. Ranjithkumar S
------------	--------------------

1. Project Title

MatrixLang: A Dimension-Aware Optimizing Compiler for a Matrix Language

2. Abstract

MatrixLang is a small language for matrix computation. Its type system carries matrix dimensions: a 2 by 3 matrix has the type $\text{Matrix}\langle 2 \times 3 \rangle$, so the compiler rejects a product with mismatched inner dimensions before it computes any element. In Phase 1 I specified the language, designed the compiler and built a front end prototype that classified tokens and reported a syntax verdict.

Phase 2 builds the core of the compiler. A Flex scanner recognises each token class and reports lexical errors at their line and column. A Bison LALR(1) parser builds an abstract syntax tree and recovers at statement boundaries, so one run reports all the syntax errors in a file. The symbol table, a hash table, records the rows and columns of each name. The semantic analyser infers the shape of each expression and checks it against the rules of matrix algebra; when it rejects one, it names both operands, their shapes and the broken rule. A generator then lowers the checked tree to three-address code in which each temporary carries its shape. One collector prints the diagnostics of all phases in source order, and each stage prints its output on request.

The six components the Phase 1 plan assigned to this phase all work, and the command `make demo2` demonstrates them. The build produces no warnings under `-Wall -Wextra`, Bison reports no unresolved conflicts, and the 49 checks on these components pass, along with the other 96 checks in the suite.

Keywords: lexical analysis, LALR(1) parsing, abstract syntax trees, symbol tables, semantic analysis, shape inference, three-address code, error recovery.

3. Core Implementation Modules

Section 9.3 of the Phase 1 document assigned six components to Phase 2: the Flex lexer, the Bison parser, the syntax tree, the symbol table with shapes, dimension checking and three-address code. Each is a module with its own header, and each consumes the output of the stage before it.

```
source -> tokens -> syntax tree -> symbol table + shape checking -> three-address code
matrix.l matrix.y symtab.c semantic.c tac.c
tokens.c ast.c types.c
```

Module	File, under src/	Lines	Responsibility
Scanner	frontend/matrix.l	119	Keywords, names, numbers, lexical errors
Token recorder	frontend/tokens.c	65	Keeps each token for the token table
Parser	frontend/matrix.y	201	LALR(1) grammar; builds the tree; recovery
Syntax tree	frontend/ast.c	206	Nodes, tree printer, expression renderer
Shape rules	analysis/types.c	108	The types and all shape rules
Symbol table	analysis/symtab.c	124	Names with shapes, positions, usage
Semantic analyser	analysis/semantic.c	465	Resolution, shape inference, checking
Intermediate code	ir/tac.c	359	Three-address code, typed temporaries
Diagnostics	support/diag.c	133	One collector for all errors, sorted
Driver	main.c	320	Options, stage selection, exit status
Total		2100	Headers and generated C excluded

Table 1. The Phase 2 modules, their sources and what each is responsible for.

Figure 1 repeats the architecture from the Phase 1 design. Phase 2 covers the first four stages of the central column, from lexical analysis to intermediate code, plus the token table, syntax tree and symbol table on the right and the diagnostics collector and driver on the left.

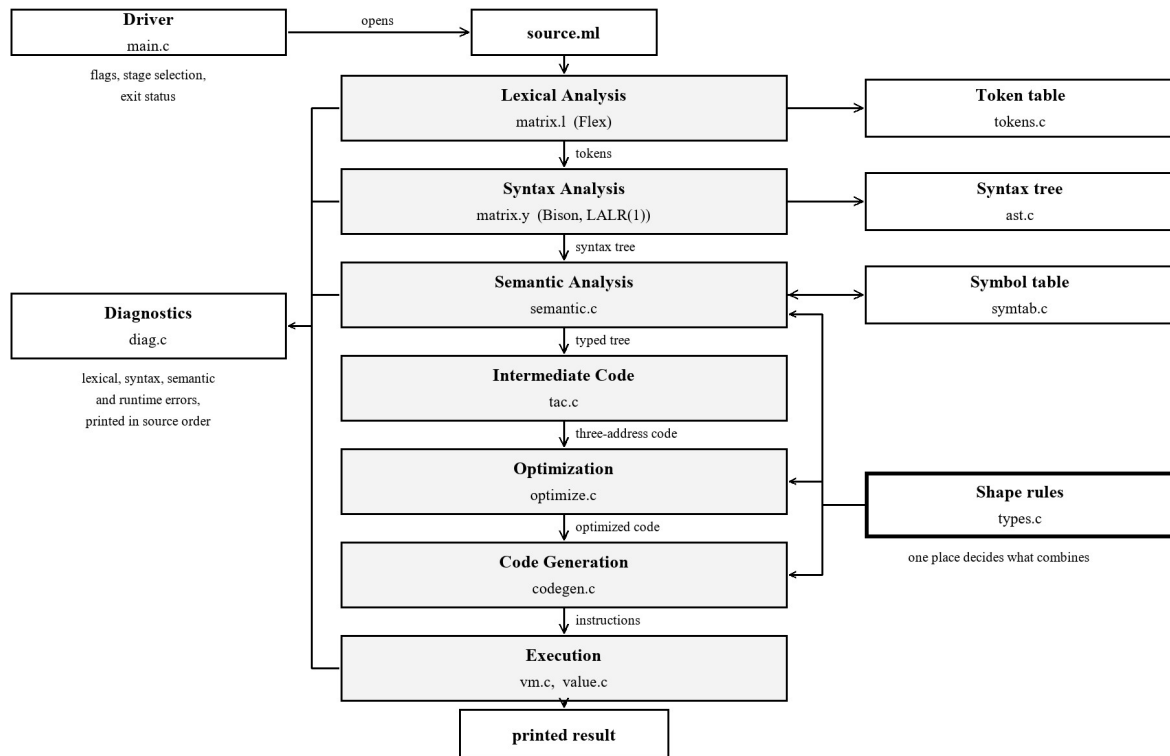


Figure 1. Architecture of the MatrixLang compiler. Phase 2 implements the stages from lexical analysis to intermediate code and the components beside them.

4. Working Source Code

4.1 Repository and organisation

I keep the source under Git and publish it at github.com/aswanth-07/matrixlang. The directories follow the phases of the compiler, so a file's directory names the part of the course it belongs to.

src/	
main.c	driver: options, stage selection, exit status
frontend/	matrix.l, matrix.y, ast.c, tokens.c
analysis/	types.c (the shape rules), syntab.c, semantic.c
ir/	tac.c, and the Phase 3 optimizer
backend/	Phase 3: code generation and the virtual machine
support/	diag.c, util.c
examples/valid/	programs that must be accepted
examples/errors/	programs that must be rejected, each saying why
demos/phase2.sh	the Review 2 demonstration
tests/run_tests.sh	the acceptance suite

4.2 Building and running

One command builds the compiler from a clean tree. Bison and Flex generate the parser and scanner into build/, and gcc compiles the hand-written sources as C11 [5].

```

$ make          # bin/matrixc
$ make test     # the acceptance suite
$ make demo2    # the Review 2 demonstration

$ ./bin/matrixc --phase2 examples/valid/multiply.ml
  
```

The compiler selects its stages by option, so a reviewer can stop it after any stage and read that stage's output. Table 2 lists the options for this phase.

Option	Shows
--phase2	Each Phase 2 stage in order: tokens, tree, symbols, diagnostics, TAC
--tokens	The token stream, classified and located by line and column
--ast	The syntax tree, each node annotated with its inferred shape
--symbols	The symbol table, with rows and columns
--check	All diagnostics and the accept or reject verdict
--tac	Three-address code, each instruction with the shape it produces
--stats	Counts of tokens, tree nodes, symbols and instructions

Table 2. Command-line options that expose the Phase 2 stages.

4.3 Code quality

- **Warning free.** The hand-written sources compile under -std=c11 -Wall -Wextra with no warnings. The Makefile compiles the generated scanner and parser without -Wextra, because Flex and Bison emit code that trips warnings this project cannot fix.
- **Modular structure.** Each module keeps its tables private to its own file and exposes an interface through its header. The symbol table, the instruction array and the diagnostic list each belong to one module.
- **Comments record decisions.** Each file opens with a comment stating its purpose. Comments inside the code give the reason for a choice, such as the rejection of scalar plus matrix, and leave the mechanics to the code.
- **No third-party libraries.** I wrote everything beyond Flex, Bison and the C standard library for this project.

5. Functional Demonstration

The manual requires a working demonstration for Review 2. The command `make demo2` runs the modules on two programs that differ in one dimension: the compiler accepts the first and lowers it to three-address code, and refuses the second. The listings below omit the banner rules; the remaining text is the compiler's output.

5.1 An accepted program

```
$ cat examples/valid/multiply.ml
/* multiply.ml -- the headline example.
   A is 2x3 and B is 3x4, so A * B is 2x4 and the compiler works that out
   rather than being told. */

matrix A[2,3] = {{1, 2, 3},
                 {4, 5, 6}};

matrix B[3,4] = {{1, 0, 0, 1},
                 {0, 1, 0, 2},
                 {0, 0, 1, 3}};

matrix C = A * B;      // shape inferred: Matrix<2x4>

print(C);
```

```
$ ./bin/matrixc --phase2 examples/valid/multiply.ml

PHASE 1 -- LEXICAL ANALYSIS (token stream)
... 78 token(s), shown in Section 7.1 ...
PHASE 1/2 -- SYNTAX ANALYSIS (abstract syntax tree)
... shown in Section 7.2 ...
PHASE 2 -- SYMBOL TABLE
... 3 symbol(s), shown in Section 7.3 ...
PHASE 2 -- SEMANTIC ANALYSIS (dimension checking)
No diagnostics. The program is valid MatrixLang.
PHASE 2 -- INTERMEDIATE CODE (three-address code)
... 5 instruction(s), shown in Section 7.4 ...

examples/valid/multiply.ml: ACCEPTED (0 error(s), 0 warning(s))
```

The declaration of C gives no dimensions. The compiler infers from the shapes of A and B that C is a 2 by 4 matrix, records that shape in the symbol table and carries it into the intermediate code. Section 7 prints each of these artefacts in full.

5.2 The same program with one dimension changed

The second program changes the shape of B to 5 by 4. A stays 2 by 3, so the columns of A no longer match the rows of B and the product has no definition.

```
$ cat examples/errors/mul_mismatch.ml
/* mul_mismatch.ml -- the Review 2 rejection demo.
   B was 3x4 in multiply.ml. Making it 5x4 breaks columns(A) == rows(B),
   and the compiler must say so before any element is multiplied. */

matrix A[2,3];
matrix B[5,4];
matrix C[2,4];

C = A * B;

print(C);

$ ./bin/matrixc --check examples/errors/mul_mismatch.ml

9:7: error [semantic] cannot multiply Matrix<2x3> by Matrix<5x4>
      left   : A -> Matrix<2x3>
      right  : B -> Matrix<5x4>
      rule   : columns(left) must equal rows(right)
      found  : 3 != 5

1 error(s), 0 warning(s).

examples/errors/mul_mismatch.ml: REJECTED (1 error(s), 0 warning(s))
$ echo $?
1
```

The error points at line 9, column 7, the position of the multiplication operator. The compiler exits with status 1 and produces no intermediate code. The driver skips code generation for any program with errors, since code for this one would contain a multiplication no machine can perform.

6. Test Cases

6.1 Method

The acceptance suite, `tests/run_tests.sh`, runs the compiler on the example programs and checks two things per case: the exit status and specific text in the output. A check on the exit status alone would pass a compiler that rejected a program for the wrong reason. A check on the text alone would pass one that printed the right message and exited with status 0.

6.2 The Phase 2 test cases

Test section	What it asserts	Checks
Syntax tree carries shapes	BinaryOp * : Matrix<2x4>; Transpose : Matrix<3x2>	6
Symbol table rows and columns	Rows and Cols present; symbol count; both kinds	7
Multiplication rule	Exit 1; the message, the rule and 3 != 5	4
Addition rule	Exit 1; the message and 2x3 against 3x2	3
Remaining shape rules	Scalar transpose, A + s, two bad dimensions	5
Matrix literals	Ragged row; literal shape against declaration	3
Declaration and name errors	Undeclared, duplicate; syntax and lexical errors	13
Three-address code	Temporaries, copies, print; shapes in the IR	8
Total		49

Table 3. The sections of the suite that test Phase 2, and how many checks each makes.

6.3 Invalid inputs

Each program in examples/errors/ exercises one family of error and opens with a comment naming the error it should trigger. The compiler rejects all eight with exit status 1 and accepts the four programs in examples/valid/ with exit status 0.

Program	Class	First diagnostic reported	Errors
lexical.ml	lexical	5:7 illegal character '\$'	4
syntax.ml	syntax	6:1 unexpected IDENT, expecting ';' or '='	3
undeclared.ml	semantic	5:1 assignment to undeclared variable 'B'	2
duplicate.ml	semantic	4:8 duplicate declaration of 'A'	2
mul_mismatch.ml	semantic	9:7 cannot multiply Matrix<2x3> by Matrix<5x4>	1
add_mismatch.ml	semantic	6:14 matrix addition requires identical dimensions	1
bad_shape.ml	semantic	9:15 transpose() expects a matrix, got Scalar	4
bad_literal.ml	semantic	5:18 row 2 of this matrix literal has 2 entries, expected 3	2

Table 4. The invalid test programs and what the compiler reports for each.

The file lexical.ml also produces two syntax errors: each illegal lexeme leaves a gap that the parser trips over. The compiler reports all four in one run, in line order; Section 8.7 explains the ordering.

6.4 Results

```
$ make test

--- Phase 2 -- the AST carries inferred shapes
PASS multiply.ml --ast (exit 0)
PASS ...contains: BinaryOp * : Matrix<2x4>
PASS ...contains: Identifier A : Matrix<2x3>
PASS ...contains: Identifier B : Matrix<3x4>
PASS transpose.ml --ast (exit 0)
PASS ...contains: Transpose : Matrix<3x2>

... the remaining sections elided ...

=====
145 passed, 0 failed
=====
```

Of the 145 checks, 27 cover the Phase 1 front end, 49 the Phase 2 components above, 64 the Phase 3 stages and 5 the driver's option handling. All 145 pass on the current tree.

7. Intermediate Results

Sections 7.1 to 7.4 trace multiply.ml from Section 5.1 through each stage, with the artefact each stage hands to the next. Section 7.5 traces shape inference through longer expressions.

7.1 Token stream

```
$ ./bin/matrixc --tokens examples/valid/multiply.ml
```

#	TOKEN	LEXEME	LINE:COL
1	MATRIX	matrix	5:1
2	IDENTIFIER	A	5:8
3	LBRACKET	[	5:9
4	NUMBER	2	5:10
5	COMMA	,	5:11
6	NUMBER	3	5:12
7	RBRACKET	]	5:13
... tokens 8 to 66 elided ...			
67	MATRIX	matrix	12:1
68	IDENTIFIER	C	12:8
69	ASSIGN	=	12:10
70	IDENTIFIER	A	12:12
71	MULTIPLY	*	12:14
72	IDENTIFIER	B	12:16
73	SEMICOLON	;	12:17
... tokens 74 to 78 elided ...			
78 token(s).			

The header comment on lines 1 to 3 and the comment at the end of line 12 produce no tokens, and positions count from the first line of the file. A dimension error on the multiplication operator would report its recorded position, 12:14.

7.2 Annotated syntax tree

```
$ ./bin/matrixc --ast examples/valid/multiply.ml
```

```
Program (line 1)
-- Declare A : Matrix<2x3> (line 5)
  -- MatrixLiteral : Matrix<2x3> (line 5)
    -- Row : Matrix<1x3> (line 5)
      -- Number 1 : Scalar (line 5)
      -- Number 2 : Scalar (line 5)
      -- Number 3 : Scalar (line 5)
    -- Row : Matrix<1x3> (line 6)
      ... three entries elided ...
  -- Declare B : Matrix<3x4> (line 8)
    -- MatrixLiteral : Matrix<3x4> (line 8)
      ... three rows elided ...
  -- Declare C : Matrix<2x4> (line 12)
    -- BinaryOp * : Matrix<2x4> (line 12)
      -- Identifier A : Matrix<2x3> (line 12)
      -- Identifier B : Matrix<3x4> (line 12)
    -- Print : Matrix<2x4> (line 14)
      -- Identifier C : Matrix<2x4> (line 14)
```

The type Matrix<2x4> on the multiplication node and on the declaration of C appears nowhere in the source. The parser built the tree without types, and the semantic analyser attached a type to each node afterwards, so those two annotations show shape inference at work. The full tree has 34 nodes and a height of 5.

7.3 Symbol table

```
$ ./bin/matrixc --symbols examples/valid/multiply.ml
```

Name	Kind	Rows	Cols	Scope	Decl@Ln	Writes	Reads
A	Matrix	2	3	global	5	1	1
B	Matrix	3	4	global	8	1	1
C	Matrix	2	4	global	12	1	1

```
3 symbol(s).
```

The Rows and Cols columns hold the shape information. Each later check on an identifier reads its shape from this table, the inferred shape of C included.

7.4 Three-address code

```
$ ./bin/matrixc --tac examples/valid/multiply.ml

1  A = #0                Matrix<2x3>
2  B = #1                Matrix<3x4>
3  t1 = A * B            Matrix<2x4>
4  C = t1                Matrix<2x4>
5  print C               Matrix<2x4>

5 instruction(s).
```

The operands #0 and #1 name the two matrix literals, which the analyser stored in a literal pool while checking them. The temporary t1 holds the product, and the right-hand column gives the shape each instruction produces.

7.5 Shapes through longer expressions

In chain.ml, A is 2 by 3, B is 3 by 2 and C is 2 by 2, and the program computes $R = A * B + C - C$. Each operator becomes one instruction with a new temporary, and the compiler infers the shape of each temporary from its operands.

```
$ ./bin/matrixc --tac examples/valid/chain.ml

1  A = #0                Matrix<2x3>
2  B = #1                Matrix<3x2>
3  C = #2                Matrix<2x2>
4  t1 = A * B            Matrix<2x2>
5  t2 = t1 + C            Matrix<2x2>
6  t3 = t2 - C            Matrix<2x2>
7  R = t3                Matrix<2x2>
8  print R               Matrix<2x2>

8 instruction(s).
```

The operator * denotes three operations, and the operand types decide which one applies. Table 5 lists the types the compiler infers across the valid examples.

Expression	Operand types	Inferred type	Meaning
A * B	Matrix<2x3>, Matrix<3x4>	Matrix<2x4>	matrix product
k * M	Scalar, Matrix<2x2>	Matrix<2x2>	scaling
M * half	Matrix<2x2>, Scalar	Matrix<2x2>	scaling
k * half	Scalar, Scalar	Scalar	scalar product
transpose(A)	Matrix<2x3>	Matrix<3x2>	rows and columns exchanged
A * transpose(A)	Matrix<2x3>, Matrix<3x2>	Matrix<2x2>	matrix product

Table 5. Types inferred for expressions in multiply.ml, scalars.ml and transpose.ml.

Each instruction keeps its type, so the Phase 3 code generator can pick a different machine instruction for each meaning of * without recomputing a shape.

8. Module-wise Explanation

8.1 Lexical analysis: matrix.l and tokens.c

Flex [4] generates the scanner from regular definitions and a list of rules, each a regular expression paired with an action [1], [2]. The definitions below describe identifiers and the two forms of numeric literal; a real literal may carry a fractional part, an exponent or both.

```
DIGIT    [0-9]
LETTER   [A-Za-z_]
IDENT    {LETTER}{LETTER}{DIGIT})*
INTLIT   {DIGIT}+
REALLIT  ({DIGIT}+"."{DIGIT}*|"."{DIGIT}+)([eE][+-]?{DIGIT}+)?|{DIGIT}+[eE][+-]?{DIGIT}+
```

Class	Members	Token names
Keywords	matrix scalar print transpose identity zeros ones	MATRIX, SCALAR, PRINT, ...
Identifiers	a letter or underscore, then letters, digits, underscores	IDENTIFIER
Numbers	2 0.5 .5 1e3 2.5E-2	NUMBER
Operators	+ - * =	PLUS, MINUS, MULTIPLY, ASSIGN
Delimiters	[] () { } , ;	LBRACKET, RBRACKET, ..., SEMICOLON
Skipped	whitespace, // line comments, /* block comments */	none

Table 6. The token classes the scanner recognises.

Keyword recognition. Flex settles competing rules by longest match and breaks ties by rule order. For the input `matrix`, the keyword rule and the identifier rule both match six characters; the keyword rule comes first and wins. For `matrixx`, the identifier rule matches seven characters against the keyword's six, so the scanner returns an identifier. The scanner needs no keyword lookup table.

Lexical errors. Three rules report errors: one for an unterminated block comment, one for an identifier that begins with a digit, and a final catch-all for any character the other rules reject. Longest match motivates the second rule. Without it, Flex would scan `12abc` as the number `12` followed by the identifier `abc`, and the parser would report a confusing syntax error. With it, the whole lexeme forms one match and the scanner reports the real mistake. After an error the scanner continues, so it finds later errors too.

Position tracking. The macro `YY_USER_ACTION` runs before each rule's action. It writes the start line and column into the location Bison reads, then calls `lex_advance`, which walks the matched text: a newline moves to the next line, a tab advances four columns and any other character one. All matches pass through this one function, so a block comment that spans several lines leaves the position correct without a separate start condition.

Token recording. Bison pulls tokens one at a time and discards each after shifting it, so no stream remains to display after the parse. A second scan of the file would report each lexical error twice. The scanner appends each token to the list in `tokens.c` as it returns it, and the token table comes out of the parse at no extra cost.

8.2 Syntax analysis: `matrix.y`

Bison [3] generates an LALR(1) parser [1] from the grammar in the Phase 1 document. The expression rules keep their natural, ambiguous form, and precedence and associativity declarations make them deterministic, so the grammar needs no separate term and factor levels.

```
%left '+' '-'
%left '*'
%right UMINUS

expr : expr '+' expr | expr '-' expr | expr '*' expr
    | '-' expr %prec UMINUS
    | ...
```

Bison builds 86 LALR(1) states for this grammar. Its report lists twelve shift/reduce conflicts in the expression rules and resolves all twelve with these declarations. For `A + B * C`, the parser reaches a state holding `expr + expr` with `*` as lookahead. The `*` operator has higher precedence, so the parser shifts and groups `B * C` first. With `+` as lookahead the two operators tie and associate to the left, so the parser reduces. The build passes -Wcounterexamples to Bison, which makes any future conflict arrive with an example input.

The semantic actions build the tree and nothing more. The parser cannot know the shape of an identifier, and giving it that job would tangle two phases. Each declaration form records the shape written in the source or, for matrix $C = A * B$, marks the shape for inference.

Error recovery. One production gives statement-level recovery. On a syntax error the parser pops states until one can shift the special error token, discards input up to the next semicolon and resumes. The call to `yerror` tells Bison that recovery has finished, so Bison reports the next error it meets.

```
stmt : decl | assign | print_stmt | ';'
    | error ';' { $$ = node_new(N_EMPTY, ...); yerror; }
```

The directive `parse.error verbose` makes each message list the tokens the parser would have accepted; `syntax.ml` shows this as expecting `';` or `'='`. Recovery needs care with memory, because discarded symbols do not reach the action that would free them. A destructor frees discarded identifier strings. The grammar declares no destructor for tree nodes: Bison destroys whatever remains on its stack after a successful parse too, and a node destructor would free the finished tree.

8.3 Abstract syntax tree: `ast.c`

All nodes share one structure: a kind tag, a source position and a growable vector of children. Four traversals walk the tree: semantic analysis, the printer, code generation and the expression renderer. With a uniform node, each traversal is a single switch over the kind.

Field	Purpose
kind	One of 15 node kinds, from <code>N_PROGRAM</code> and <code>N_DECL</code> to <code>N_NUMBER</code> and <code>N_IDENT</code>
line, col	Source position, reported by each diagnostic about the node
name, dval	Identifier or operator spelling; the value of a number
decl_type, has_dims	Declarations only: the shape written in the source, or a mark for inference
type	The shape inferred by semantic analysis
lit_id, d1, d2	Literal pool index; folded dimensions of identity, zeros and ones
kids, nkids	The children, in source order

Table 7. Fields of the syntax tree node.

I added two features for the diagnostics. A matrix literal keeps its row structure in the tree, so the analyser can say which row is short. The function `ast_expr_text` renders any expression back into source text, so a dimension error names an operand as `left : A * B`. Error recovery leaves an empty node in place of a discarded statement, and the tree stays well formed after an error.

8.4 Symbol table: `syntab.c`

The symbol table is a hash table of 211 buckets (a prime) that uses the djb2 string hash and separate chaining. Insertion and lookup take constant time on average. A second list links the symbols in insertion order, because the table prints in declaration order and hashing scrambles that order.

```
static unsigned hash(const char *s)          /* djb2: h = h * 33 + c */
{
    unsigned h = 5381u;
    while (*s) h = ((h << 5) + h) + (unsigned char)*s++;
    return h % NBUCKETS;
}
```

Each entry records the name, its type (for a matrix, the rows and columns), the line and column of its declaration, whether any statement reads it, and counts of reads and writes. Insertion refuses a duplicate name and returns no symbol; the caller looks up the earlier entry to report its line. The check spans kinds, so scalar `x` followed by matrix `x` counts as a duplicate.

The table has one scope. MatrixLang has no blocks and no functions, a decision recorded in Section 6.3 of the Phase 1 document, so a scope stack would hold a single entry. The Scope column reads global on each row for that reason.

8.5 Semantic analysis: semantic.c and types.c

The analyser makes one pass over the tree. It checks statements in order; for each expression, a recursive function computes the type from the children's types and stores it on the node. The type is a synthesized attribute in the sense of syntax-directed definitions [1], evaluated bottom-up.

Operation	Operand shapes	Result	Rejected when
$A + B, A - B$	$r \times c$ and $r \times c$	$r \times c$	shapes differ, or one operand is a scalar and the other a matrix
$s + t, s - t$	scalar and scalar	scalar	no restriction
$A * B$	$r \times n$ and $n \times c$	$r \times c$	columns of A differ from rows of B
$s * A, A * s$	scalar and $r \times c$	$r \times c$	no restriction (scaling)
$s * t$	scalar and scalar	scalar	no restriction
$\text{transpose}(A)$	$r \times c$	$c \times r$	the operand is a scalar
$-A$	any	unchanged	no restriction
$\text{identity}(n)$	constant n	$n \times n$	n is not a constant positive whole number
$\text{zeros}(r, c),$ $\text{ones}(r, c)$	constants r, c	$r \times c$	r or c is not a constant positive whole number

Table 8. The shape rules, all implemented in types.c.

The file types.c holds these rules as functions such as `type_mul`, which returns the product's type or the error type. The analyser calls these functions and the later phases consult the same file, so one place in the compiler decides whether $A * B$ is legal and what shape it produces. I made the language reject scalar plus matrix on purpose. That addition would have to add the scalar to each element, an operation outside matrix algebra, and accepting it would let shape mistakes survive to run time.

Declarations. A declaration that gives a shape and an initialiser needs an initialiser of that shape, and an assignment must match its variable's shape too. A declaration without a shape, as in `matrix C = A * B`, takes its shape from the initialiser; an inferred shape reaches the symbol table by this path. The analyser checks the initialiser before it inserts the name, so `matrix A = A` reports A as undeclared. Declared dimensions must be at least 1.

Compile-time constants. The compiler must know each shape during compilation. A small evaluator folds numbers under negation, addition, subtraction and multiplication. Identifiers do not fold, because the language has no named constants. The arguments of `identity`, `zeros` and `ones` must fold, as must the entries of a matrix literal, and a dimension must be a positive whole number as well. For that reason `bad_shape.ml` fails on `zeros(s, 2)` and on `identity(2.5)`.

Matrix literals. The analyser checks a literal's shape before its entries. The first row sets the width, and the analyser reports any later row of a different length by its number. It folds the entries into the literal pool once the literal proves rectangular; entry errors on a literal with no shape would add noise.

Error containment. The analyser gives an expression that fails a rule the error type, and each rule accepts an operand of that type without comment, since the analyser has reported it. Without this, one undeclared name deep in an expression would trigger a further error at each operator above it. The program below has one mistake under two more operators and produces one diagnostic.

```
matrix A[2,2];
matrix R = transpose(Q) * A + A;
print(R);

2:22: error [semantic] use of undeclared variable 'Q'
1 error(s), 0 warning(s).
```

After the walk, the analyser warns about each name that no statement reads. Warnings leave the exit status unchanged. The driver skips semantic analysis for a program with lexical or syntax errors, because a tree with statements removed by recovery would yield errors that the recovery caused. The driver's output reports a skipped pass.

8.6 Intermediate code: tac.c

The intermediate representation is three-address code [1]: each instruction applies at most one operator to at most two operands and writes at most one destination. Ten instruction forms cover the language.

Form	Opcodes
$x = y + z$, $x = y - z$, $x = y * z$	TAC_ADD, TAC_SUB, TAC_MUL
$x = -y$, $x = \text{transpose}(y)$	TAC_NEG, TAC_TRANS
$x = y$	TAC_COPY
$x = \text{identity}(n)$, $x = \text{zeros}(r,c)$, $x = \text{ones}(r,c)$	TAC_IDENTITY, TAC_ZEROS, TAC_ONES
print y	TAC_PRINT

Table 9. The instruction forms of the three-address code.

Generation is a postorder walk. For each expression, the generator returns the name of the operand holding its value: a number becomes a constant operand, an identifier names itself, a matrix literal becomes its index in the literal pool, and each operator writes a new temporary. A declaration without an initialiser emits nothing, because each declared variable starts at zero with its declared shape. Operands are strings, and the spelling of each one tells its kind.

```
A, C      a program variable (an identifier never starts with a digit)
t1, t7    a compiler temporary
3, 2.5    a scalar constant
#0, #4    a matrix literal, by index into the literal pool
```

The generator interns each operand string, storing it once and sharing it. Two operands are equal when their strings match, and no instruction owns an operand it must free, so later passes can copy operands between instructions. Each instruction records the type it produces, and the generator types each temporary as it creates it. A MatrixLang program has no control flow and forms a single basic block, so one flat array of instructions represents it.

8.7 Error handling: diag.c

The phases print no messages of their own. Each reports through one function that records the level, phase, position and text, and may attach a detail block of several lines. Four phases can report, and each message is an error or a warning.

Class	Detected by	Example
Lexical	Scanner	illegal character '\$'
Syntax	Parser	unexpected ';', expecting ']'
Semantic	Semantic analyser	cannot multiply Matrix<2x3> by Matrix<5x4>
Runtime	Virtual machine (Phase 3)	virtual machine stack overflow

Table 10. Error classes and the component that detects each.

The collector sorts the messages by line and column before printing them, because the phases do not run in source order. In lexical.ml the scanner's errors and the parser's errors interleave, and the sort prints them in file order.

```

5:7: error [lexical] illegal character '$'
5:9: error [syntax] syntax error, unexpected IDENT, expecting '+' or '-' or '*' or ';'
6:5: error [lexical] malformed number or identifier '12abc' (an identifier may not begin with
a digit)
6:11: error [syntax] syntax error, unexpected '*'
4 error(s), 0 warning(s).

```

The error count sets the exit status: 0 for a valid program, 1 when the compiler reported an error, and 2 for a usage problem such as a missing file. A dimension error carries a detail block with both operands as written, their shapes, the rule and the values found, as in Section 5.2.

9. Implementation Progress Report

9.1 Status against the plan

Planned component	Status	Shown by	Tested by
Flex lexer	Complete	--tokens	token class checks; lexical.ml
Bison parser, error recovery	Complete	--ast	syntax.ml: three errors in one run
Syntax tree with shapes	Complete	--ast	6 checks
Symbol table with shapes	Complete	--symbols	7 checks
Dimension checking	Complete	--check	28 checks over eight invalid programs
Three-address code	Complete	--tac	8 checks
Review 2 demonstration	Complete	make demo2	the runs of Section 5

Table 11. The Phase 2 plan from the Phase 1 document, and the state of each item.

9.2 Technical challenges addressed

- **A token table from a pull parser.** Bison consumes tokens as it parses, which leaves nothing to print afterwards. The scanner records each token as it returns it, which gives the table without a second scan or duplicated lexical errors.
- **Error containment.** Left alone, a bad operand deep in an expression produces an error at each operator above it. The error type, which each rule accepts without comment, cuts that down to one message.
- **Messages in file order.** Different phases find errors, and they do not run in file order. A single collector that sorts by position fixes the order for all phases at once.
- **Memory during error recovery.** Symbols that recovery discards do not reach the action that frees them. A destructor frees discarded identifier strings. A destructor for tree nodes would free the finished tree after a successful parse, so the grammar omits it and records the reason beside the declaration.
- **A silent build failure on Windows.** With a conflicting runtime library earlier on PATH, gcc exits with status 1 and prints nothing. The build instructions put the mingw64 tools first, and make toolchain prints the tools the build resolves. gcc also falls back to C:\Windows for temporary files, a directory it cannot write to, so the Makefile points it at build/.

9.3 Distinctive features

- **Shapes are types.** Matrix<2x3> and Matrix<3x2> are different types, and a declaration may leave its shape to the compiler.
- **Diagnostics that state the rule.** A rejection names the operands as written, their shapes, the broken rule of matrix algebra and the numbers that broke it.
- **A typed intermediate representation.** Each instruction and temporary carries its shape, ready for the instruction selection and cost model of Phase 3.
- **Visible stages.** Each stage prints its artefact on request, so a reviewer can stop the compiler after any stage and read its output.

- **A visual demonstration of the front end.** A web page steps through the scanner and the LALR(1) parse of any program you type into it. It shows the competing scanner rules, the parser's stack and lookahead, each action, and the tree as it forms, using the state table from Bison's own report. A check that runs with the tests compares the page's scanner and parser with the compiler over the 21 example programs, and the two agree on all 1,179 tokens and 10 front end diagnostics.

9.4 Work for Review 3

Phase 3 covers the optimizer (algebraic simplification over matrix properties, common subexpression elimination, copy propagation, dead code elimination, and ordering of matrix chains by arithmetic cost), target code generation for a stack machine, execution with runtime errors, and a measured evaluation. The current source tree contains these stages, and they pass the suite's 64 Phase 3 checks. Review 3 will present them with the evaluation.

References

- [1] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. Compilers: Principles, Techniques, and Tools. 2nd edition, Pearson Addison Wesley, 2006.
- [2] Levine, J. flex & bison: Text Processing Tools. O'Reilly Media, 2009.
- [3] Free Software Foundation. Bison: The Yacc-compatible Parser Generator, version 3.8.2. <https://www.gnu.org/software/bison/manual/>
- [4] The Flex Project. Lexical Analysis with Flex, version 2.6.4. <https://westes.github.io/flex/manual/>
- [5] ISO/IEC 9899:2011. Information technology, programming languages, C. International Organization for Standardization, 2011.